



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 05

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 06

GRUPO DE TEORÍA: 03

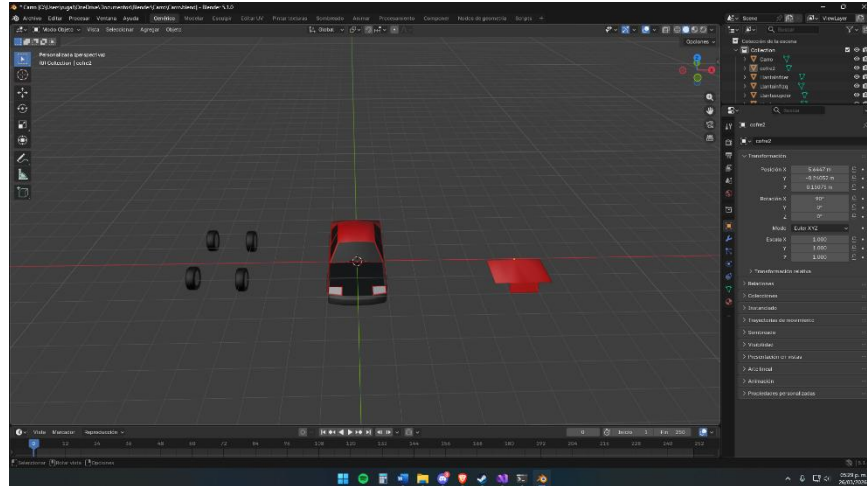
SEMESTRE: 2026-2

FECHA DE ENTREGA LÍMITE: 29/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.



- Importar su modelo de coche propio dentro del escenario a una escala adecuada.
- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.
- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.
- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

Variables de control y constantes de animación

```
// Llantas giro libre 360, cofre 0-45
const float LIMITE_LLANTA = 360.0f;
const float LIMITE_COFRE = 45.0f;

float angLlantaSupDer = 0.0f;
float angLlantaSupIzq = 0.0f;
float angLlantaInfDer = 0.0f;
float angLlantaInfIzq = 0.0f;
```

```
float angCofre = 0.0f;

// Traslacion global del carro (Z = adelante/atras)

float posCarroZ = 0.0f;
```

Se definen las constantes de límite para llantas (360°) y cofre (45°). Las variables flotantes almacenan el ángulo acumulado de cada llanta, el ángulo del cofre y la posición Z global del carro. Al separar estas variables se permite controlar cada articulación de forma independiente desde el teclado.

Carga de modelos OBJ

```
carro.LoadModel("Models/Carro.obj");
cofre.LoadModel("Models/Cofre.obj");
llanta_infder.LoadModel("Models/Llanta_infder.obj");
llanta_infizq.LoadModel("Models/Llanta_infizq.obj");
llanta_supder.LoadModel("Models/Llanta_supder.obj");
llanta_supizq.LoadModel("Models/Llanta_supizq.obj");
```

Se cargan en memoria los seis modelos OBJ del vehículo: el cuerpo principal, el cofre y las cuatro llantas. Cada pieza fue exportada por separado para permitir aplicarle transformaciones jerárquicas independientes sin afectar al resto del ensamble.

Función handleModelKeys — control de teclado

```
void handleModelKeys(bool* keys, float dt)
{
    float velLlanta = 90.0f * dt;
    float velCofre = 60.0f * dt;
    float velCarro = 5.0f * dt;

    // Avanzar/retroceder carro: Z adelante, X retrocede
```

```

    if (keys[GLFW_KEY_Z]) posCarroZ -= velCarro;
    if (keys[GLFW_KEY_X]) posCarroZ += velCarro;

    // Cofre: R abre, F cierra
    if (keys[GLFW_KEY_R]) angCofre += velCofre;
    if (keys[GLFW_KEY_F]) angCofre -= velCofre;
    angCofre = clampf(angCofre, 0.0f, LIMITE_COFRE);

    // Llanta sup der: T adelante, G atras
    if (keys[GLFW_KEY_T]) angLlantaSupDer += velLlanta;
    if (keys[GLFW_KEY_G]) angLlantaSupDer -= velLlanta;
    angLlantaSupDer = clampf(angLlantaSupDer, -LIMITE_LLANTA, LIMITE_LLANTA);

    // ... (mismo patron para las demas llantas)
}

```

Esta función lee el estado de cada tecla en cada fotograma y actualiza las variables de control multiplicando la velocidad por deltaTime, logrando movimiento independiente de la tasa de fotogramas. La tecla Z desplaza el carro hacia adelante y X hacia atrás. R/F abren y cierran el cofre (limitado entre 0° y 45°). Los pares T/G, Y/H, U/J e I/K controlan la rotación de cada llanta individualmente.

Renderizado del cuerpo principal del carro

```

// CARRO (cuerpo principal)
color = glm::vec3(0.6f, 0.6f, 0.6f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
modelaux = baseMat;
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 1.5f, 0.0f));
modelaux = glm::scale(modelaux, glm::vec3(escala, escala, escala));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
carro.RenderModel();

```

La matriz del cuerpo parte de baseMat, se eleva 1.5 unidades en Y para posicionarlo sobre el piso y se escala uniformemente con escala = 5.0. Se envía al shader mediante el uniform uniformModel y se renderiza el modelo. El orden translate → scale es correcto ya que el escalado no debe afectar el pivote de traslación.

Renderizado del cofre con rotación jerárquica

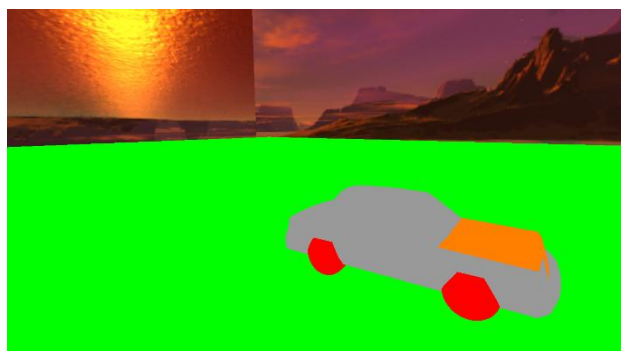
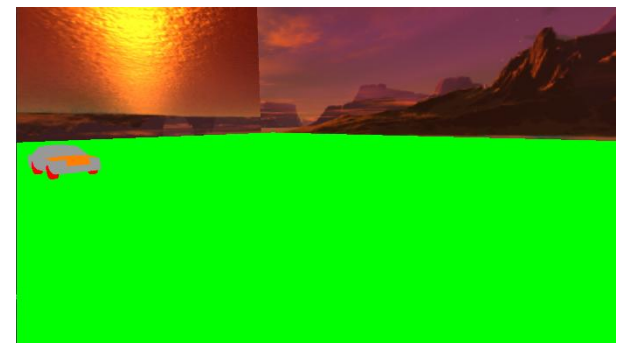
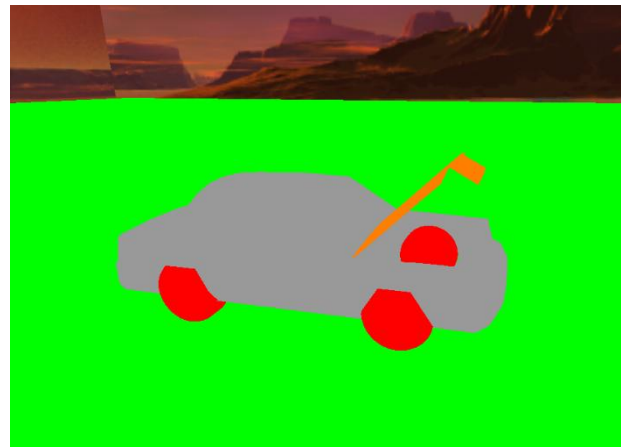
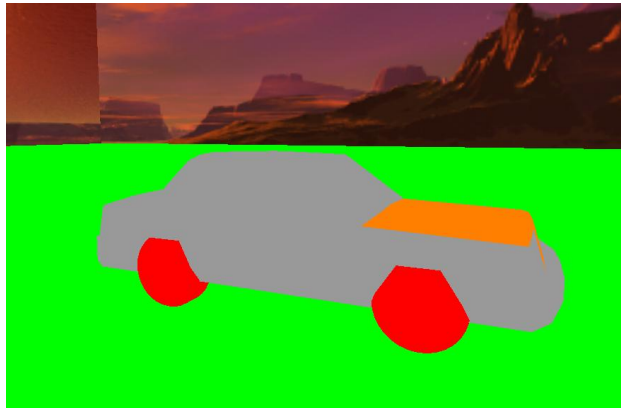
```
// COFRE - R abre (hasta 45 grados), F cierra
color = glm::vec3(1.0f, 0.5f, 0.0f);
modelaux = baseMat;
modelaux = glm::translate(modelaux, glm::vec3(-0.62f, 1.85f, -5.40f));
modelaux = glm::rotate(modelaux, angCofre * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = glm::scale(modelaux, glm::vec3(escala, escala, escala));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
cofre.RenderModel();
```

El cofre hereda baseMat y se traslada al pivote de su bisagra en el chasis. La rotación se aplica sobre el eje X con angCofre (limitado entre 0° y 45°), simulando la apertura física del cofre. El orden translate → rotate → scale es clave: si se escala antes de rotar, el pivote se desplaza y el cofre giraría fuera de posición.

Renderizado de las cuatro llantas

```
// LLANTA SUP DER - T adelante, G atras
modelaux = baseMat;
modelaux = glm::translate(modelaux, glm::vec3(4.3f, -1.1f, -7.70f));
modelaux = glm::rotate(modelaux, angLlantaSupDer * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = glm::scale(modelaux, glm::vec3(escala, escala, escala));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
llanta_supder.RenderModel();
// (mismo patron para llanta_supizq, llanta_infder, llanta_infizq)
```

Cada llanta hereda baseMat y se traslada a su posición en el chasis. La rotación sobre el eje X simula el giro de rodadura. Las cuatro llantas siguen el mismo patrón jerárquico pero con vectores de traslación y variables de ángulo propias, lo que permite rotarlas de forma individual o simultánea según las teclas presionadas.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Durante el desarrollo de la práctica se presentaron los siguientes inconvenientes:

- Posicionamiento de pivotes: al importar el cofre y las llantas, los pivotes de los modelos OBJ no coincidían con el origen del mundo, por lo que las rotaciones generaban desplazamientos no deseados. Se resolvió ajustando manualmente los vectores de traslación en baseMat antes de aplicar la rotación en la matriz jerárquica, buscando el punto que correspondía a la bisagra o al eje de cada llanta.
- Escala del modelo: el modelo del carro importado aparecía con un tamaño incorrecto respecto al plano de piso. Se determinó de forma empírica el valor escala = 5.0 probando valores hasta obtener proporciones visuales aceptables.
- Movimiento dependiente de la tasa de fotogramas: en una primera versión, el movimiento de las piezas era inconsistente entre ejecuciones. Se corrigió multiplicando todas las velocidades de rotación y traslación por deltaTime, haciendo el comportamiento uniforme en cualquier equipo.

3.- Conclusión:

La práctica abordó la carga e integración de modelos 3D externos en formato OBJ dentro de un escenario OpenGL, aplicando jerarquías de transformaciones matriciales para articular un vehículo compuesto por múltiples piezas independientes. La complejidad principal radica en comprender el orden correcto de las operaciones: la traslación al pivote debe aplicarse antes de la rotación y el escalado siempre al final, ya que invertir este orden desplaza el punto de giro y produce animaciones incorrectas. Separar cada pieza articulada en su propio modelo OBJ resultó ser la estrategia adecuada para lograr rotaciones locales sin afectar al resto del ensamble, y el uso de una matriz base compartida simplificó considerablemente la traslación global del conjunto. Sería útil que en futuras sesiones se dedicara más tiempo a la elección del pivote de rotación correcto para piezas articuladas, ya que fue el punto que generó mayor confusión durante la implementación, y que se mostrara al menos un ejemplo de depuración visual de posiciones antes de ajustar los vectores manualmente. En general, la práctica tiene un objetivo claro y el código base proporcionado fue una buena guía de partida para comprender la aplicación práctica del modelo jerárquico en visualización 3D interactiva.

Bibliografía

- Sellers, G., Wright, R. S., y Haemel, N. (2015). OpenGL superbible: Comprehensive tutorial and reference (7th ed.). Addison-Wesley.
- Shreiner, D., Sellers, G., Kessenich, J., y Licea-Kane, B. (2013). OpenGL programming guide: The official guide to learning OpenGL (8th ed.). Addison-Wesley.
- The Khronos Group. (2024). OpenGL reference pages. <https://registry.khronos.org/OpenGL-Refpages/gl4>
- Madhav, S. (2020). Game programming algorithms and techniques: A platform-agnostic approach. Pearson Education.